Exercise Sheet 03

Dalvric's Tangle

House rules.

- One file per exercise, in `my_work/`: `ex1_first_function.py`, `ex2_bonus_function.py`, …
- Type the **Given** lines first. On this sheet they are usually the lines *below* the functions: the calls. You write the functions above them.
- Run it (`python my_work/ex1_first_function.py`) and compare with the **expected output**, character by character.
- A function that prints the right thing but doesn't `return` it will fail the next exercise. Return values; print at the end.
- Hand in: commit and push to `lesson-03`, then open a pull request. Tell Lars which function you're proudest of.

Exercise 1 — Your first function

5 min · `ex1_first_function.py`

> **RAVEN #301, from Mistress Pell, Paymaster:** *"I've typed `annual / 12` into so many cells I dream about it. Orrin says you can 'give it a name'. Can you give it a name?"*

GIVEN

```
print(monthly_salary(32500))
print(monthly_salary(24000))
print(monthly_salary(41000) * 12)
```

(a) Above the Given lines, write `def monthly_salary(annual_salary):` with a one-line docstring, and `return` one twelfth, rounded to the penny.

(b) Run it. The third line is not 41000. Why not? Write the answer as a comment at the bottom. (It's not a bug in Python. It's rounding, done too early.)

EXPECTED OUTPUT

```
2708.33
2000.0
41000.04
```

Hint: In VBA you'd write `Function MonthlySalary(annual) … MonthlySalary = annual / 12 … End Function`. Python: `def`, a colon, an indented body, `return`.

Exercise 2 The napkin, as a function

10 min · `ex2_bonus_function.py`

> **RAVEN #302, from Lady Siv of the Rewards:** *"Last week's bonus rule worked. Then the Queen asked for it in three other reports, and now there are four copies of the napkin and they've started to disagree. Can there be one?"*

GIVEN

```
print(bonus_rate(14, 4))
print(bonus_rate(6, 5))
print(bonus_rate(30, 3))
print(bonus_rate(30, 2))
print(bonus(31000, 14, 4))
print(bonus(400, 3, 5))
```

(a) Write `bonus_rate(months_of_service, rating)`: nothing under 12 months, 8 % for a rating of 4 or more, 4 % for a 3, else nothing. Use `return` inside each branch: once a function returns, it's finished, so you don't even need `elif`.

(b) Write `bonus(salary, months_of_service, rating)` that *calls* `bonus_rate` rather than repeating the rules. One napkin, one place.

EXPECTED OUTPUT

```
0.08
0
0.04
0
2480.0
0
```

Exercise 3 Formatting helpers

10 min · `ex3_money.py`

> **RAVEN #303, from Queen Aldara, at dusk:** *"I liked the pretty numbers. I'd like them everywhere now. Everywhere. A."*

GIVEN

```
print(money(2708.333))
print(money(50))
print(scroll_line("Basic pay", 2708.333))
print(scroll_line("Pension", 135.42))
print(percent(0.125))
print(percent(0.125, 0))
print(percent(0.05, 2))
```

(a) `money(amount)` returns the amount with a thousands separator, 2 decimals and `cr` after it.

(b) `scroll_line(label, amount)` returns the label left-aligned in 15 characters and `money(amount)` right-aligned in 12. Call `money`; don't repeat the format.

(c) `percent(fraction, decimals=1)` has a **default**: called with one argument it shows 1 decimal. Build the format with `f"{fraction * 100:.{decimals}f}%"`, an f-string inside an f-string.

EXPECTED OUTPUT

```
2,708.33 cr
50.00 cr
Basic pay       2,708.33 cr
Pension           135.42 cr
12.5%
```

```
12%
5.00%
```

Exercise 4 One pay scroll, one function

15 min · `ex4_payslip.py`

> **RAVEN #304, from Mistress Pell:** *"The pay scroll from lesson 01 was lovely. For one subject. We have 200, and Orrin's pension rate is 8 % because he 'negotiated', and Bram's is 3 % because he 'didn't'."*

GIVEN

```
print_pay_scroll("Maren Ashdown", 32500)
print_pay_scroll("Orrin Ironfoot", 36500, pension_rate=0.08)
print_pay_scroll("Bram Kettleby", 24500, 0.03, 0.20)
```

(a) Copy your `money` and `scroll_line` from exercise 3 to the top of the file. (Next lesson but one you'll learn to import them instead.)

(b) Write `print_pay_scroll(name, annual_salary, pension_rate=0.05, tax_rate=0.20)`. Basic pay is a twelfth; pension is basic × rate; crown tax is 20 % of (basic − pension); net is what's left. It *prints* the pay scroll (and one empty line after it) rather than returning anything.

(c) Look at the three Given calls: positional, by keyword, and all positional. All three are fine. Which is clearest?

EXPECTED OUTPUT

```
PAY SCROLL: Maren Ashdown
--------------------------
Basic pay       2,708.33 cr
Pension           135.42 cr
Crown tax         514.58 cr
--------------------------
Net pay         2,058.33 cr

PAY SCROLL: Orrin Ironfoot
--------------------------
Basic pay       3,041.67 cr
Pension           243.33 cr
Crown tax         559.67 cr
--------------------------
Net pay         2,238.67 cr

PAY SCROLL: Bram Kettleby
--------------------------
Basic pay       2,041.67 cr
Pension            61.25 cr
Crown tax         396.08 cr
--------------------------
Net pay         1,584.33 cr
```

Exercise 5 Holiday allowance, properly

10 min · `ex5_holiday.py`

> **RAVEN #305, from Mistress Pell:** *"Same rules as last week: 25 days, +1 after 2 years, +3 after 5, +5 after 10, best one applies. But Westmarch is on 28 days base because of 'the 2016 thing', and the Queen keeps asking 'what will it be in 2030'."*

GIVEN

```
for text in "2024,2021,2019,2015,2009".split(","):
    print(describe(int(text)))
print(holiday_days(6, base=28))
print(years_of_service(2019, this_year=2030))
```

(a) `holiday_days(service_years, base=25)` returns the days. `years_of_service(start_year, this_year=2025)` returns the years.

(b) `describe(start_year)` returns the whole sentence, using the other two functions. Get "1 year" / "4 years" right: `unit = "year" if service == 1 else "years"` is a one-line `if`.

EXPECTED OUTPUT

```
Started 2024: 1 year of service -> 25 days holiday
Started 2021: 4 years of service -> 26 days holiday
Started 2019: 6 years of service -> 28 days holiday
Started 2015: 10 years of service -> 30 days holiday
Started 2009: 16 years of service -> 30 days holiday
31
11
```

Hint: Small functions that call each other beat one big function. Each one is easy to test on its own in the REPL: `python -i my_work/ex5_holiday.py`, then `holiday_days(2)`.

Exercise 6 What happens in a function stays in a function

10 min · `ex6_scope.py`

> **RAVEN #306, from Orrin of the Clockworks:** *"The Grimoire has 212 global variables. One is called `temp2_final`. Please explain to me, slowly, why Python won't let that happen."*

GIVEN

```
rate = 0.05


def pension(basic):
    deduction = basic * rate
    return deduction


def pension_with_rate(basic, rate):
    return basic * rate


print(pension(2000))
print(pension_with_rate(2000, 0.08))
print(rate)
```

(a) Type it exactly and run it. Before you do: what will the last line print? `pension_with_rate` was given a `rate` of 0.08. Does that change the outer `rate`?

(b) Add `print(deduction)` at the bottom. Run it. Read the error. Delete the line. Variables made inside a function live only inside it.

(c) Add a function `tidy(name)` that strips and title-cases its argument and returns it. Call it with `raw = "  orrin IRONFOOT "`, store the result in `clean`, and print both in square brackets. The original text is untouched: functions get a copy to work with.

EXPECTED OUTPUT

```
100.0
160.0
0.05
[  orrin IRONFOOT ]
[Orrin Ironfoot]
```

Exercise 7 The pay gap toolkit STRETCH

15 min · `ex7_pay_gap_functions.py`

> **RAVEN #307, from Queen Aldara:** *"The pay gap sentence was a hit with the Privy Council. Now they want it for every guild, and one of them might be negative, and Lady Siv says negative pennies 'aren't a thing'. Make them a thing. A."*

GIVEN

```
m = hourly_rate(39000, 37.5)
f = hourly_rate(26100, 30)
print(round(m, 2), round(f, 2))
print(pay_gap_sentence(m, f))
print(pay_gap_sentence(18.50, 18.90))
print(pay_gap_sentence(20, 20))
```

(a) Write four functions: `hourly_rate(annual_salary, weekly_hours)` (52 weeks), `pay_gap_percent(male, female)` (gap as % of men's rate, 1 decimal), `pennies_per_crown(male, female)` (whole pennies), and `pay_gap_sentence(male, female)`.

(b) The sentence has three cases: a positive gap ("84 pennies for every crown"), a negative gap (show it as "1.02 crowns for every crown"), and exactly zero.

EXPECTED OUTPUT

```
20.0 16.73
Women earn 84 pennies for every crown that men earn (gap: 16.3%).
Women earn 1.02 crowns for every crown that men earn (gap: -2.2%).
Women and men earn the same per hour (gap: 0.0%).
```

Bug hunt Pages from the Grimoire, page 3

15 min · `bughunt/spaghetti_bug.py`

Dalvric discovered functions late in his career, wrote three, and went back to copying incantations by hand because "they kept returning nothing". There are **three** bugs: one crashes, one makes a function hand back nothing at all, and one is a default value that quietly takes half of everyone's pay.

EXPECTED OUTPUT ONCE FIXED

```
2,708.33 cr
Pension at 5%: 135.42 cr
Pension at 8%: 216.67 cr
Basic pay       2,708.33 cr
Pension           135.42 cr
```

Commit each fix separately, with a message explaining what was wrong, like a detective's report.

More riddles. Five more torn pages wait in `bughunt/riddles/`, one bug each (the last one has two), getting harder as you go. Optional: for fast finishers, or a quiet evening.